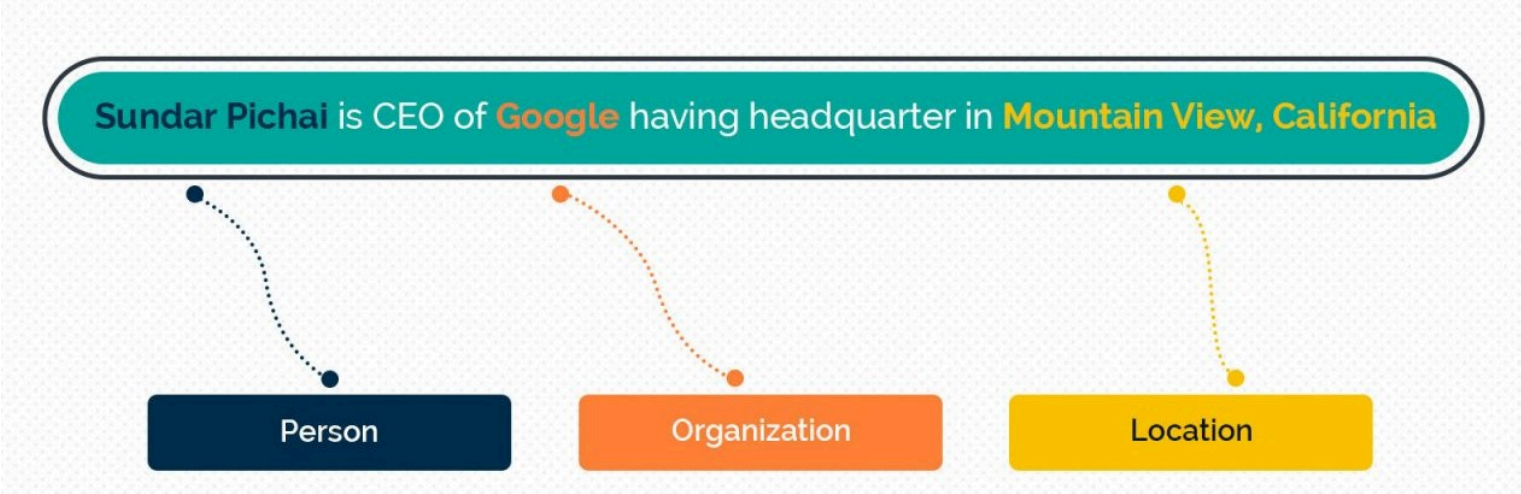
Lab 9 — Named Entity Recognition (NER)

Student Name: Faisal AL Zahrani
Student ID: 2230000363

Name Entity Recognition (NER) is a fundamental task in NLP that focuses on identifying and classifying entities such as names of people, organizations, locations, dates, and more within a given text. NER plays a crucial role in various NLP applications, including information retrieval, question answering, and sentiment analysis.

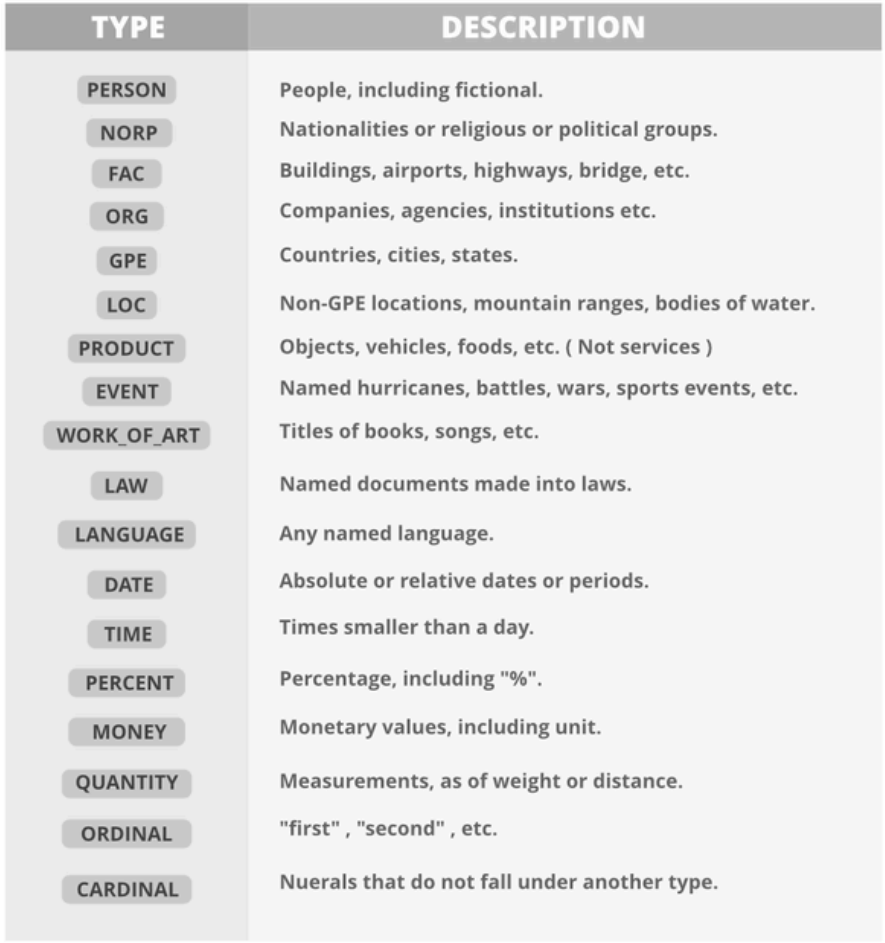


At its core, it is just a two-step process:

1. Detecting the entities from the text
2. Classifying them into different categories

NER using spaCy

SpaCy offers robust support for Named Entity Recognition (NER). It can recognize a variety of named entities, such as persons, organizations, dates, locations, and more. Here's more information about using spaCy for NER:



TYPE	DESCRIPTION
PERSON	People, including fictional.
NORP	Nationalities or religious or political groups.
FAC	Buildings, airports, highways, bridge, etc.
ORG	Companies, agencies, institutions etc.
GPE	Countries, cities, states.
LOC	Non-GPE locations, mountain ranges, bodies of water.
PRODUCT	Objects, vehicles, foods, etc. (Not services)
EVENT	Named hurricanes, battles, wars, sports events, etc.
WORK_OF_ART	Titles of books, songs, etc.
LAW	Named documents made into laws.
LANGUAGE	Any named language.
DATE	Absolute or relative dates or periods.
TIME	Times smaller than a day.
PERCENT	Percentage, including "%".
MONEY	Monetary values, including unit.
QUANTITY	Measurements, as of weight or distance.
ORDINAL	"first" , "second" , etc.
CARDINAL	Nuerals that do not fall under another type.

When performing Named Entity Recognition (NER) using spaCy or other NLP tools, you'll encounter various entity types, and they are represented as labels. Here are some common entity types and their labels:

Entity Type	Label	Description	Examples
Person	`PERSON`	Represents names of people.	*John Smith*, *Barack Obama*
Organization	`ORG`	Names of companies, institutions, agencies, or other organized entities.	*Microsoft*, *United Nations*
Location	`GPE`, `LOC`	Geographical locations such as cities, countries, or landmarks.	*New York*, *Eiffel Tower*
Date	`DATE`	Calendar dates, time intervals, or relative dates.	*January 1, 2023*, *tomorrow*
Time	`TIME`	Clock times or time intervals.	*2:30 PM*, *5 minutes*
Money	`MONEY`	Monetary values, including currencies and amounts.	*$100*, *€50*

Entity Type	Label	Description	Examples
Percent	`PERCENT`	Percentage values.	*50%, 10 percent*
Product	`PRODUCT`	Names of products or goods.	*iPhone, Coca-Cola*
Event	`EVENT`	Names of events or occurrences.	*World Cup, New Year's Eve*
Law	`LAW`	Legal references or statutes.	*Copyright Act, First Amendment*
Language	`LANGUAGE`	Names of languages.	*English, Spanish*
Nationality/Religion/Political Group	`NORP`	Nationalities, religious groups, or political affiliations.	*American, Buddhist*

In this example, we will use spaCy to perform NER on some sample text. Make sure you have spaCy installed; if not, you can install it using : `pip install spacy`

```
# Lab 9 — Named Entity Recognition (NER)
# Student Name: Faisal AL Zahrani
# Student ID: 2230000363

# Install spaCy only if missing, then load the English NER model.
import importlib.util
import subprocess
import sys

if importlib.util.find_spec('spacy') is None:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', 'spacy'])

import spacy

try:
    nlp = spacy.load('en_core_web_sm')
except OSError:
    subprocess.check_call([sys.executable, '-m', 'spacy', 'download', 'en_core_web_sm'])
    nlp = spacy.load('en_core_web_sm')

text = ('Apple Inc. was founded by Steve Jobs in Cupertino, California. '
        'It is known for its iPhone and Mac products.')
doc = nlp(text)

print('Entities recognized in the text:', [(ent.text, ent.label_) for ent in doc.ents])
for entity in doc.ents:
    print(f'Entity: {entity.text}, Label: {entity.label_}')
```

```
Entities recognized in the text: [('Apple Inc.', 'ORG'), ('Steve Jobs', 'PERSON'), ('Cupertino', 'GPE'), ('California', 'GPE'), ('Mac', 'ORG')]
Entity: Apple Inc., Label: ORG
Entity: Steve Jobs, Label: PERSON
Entity: Cupertino, Label: GPE
Entity: California, Label: GPE
Entity: Mac, Label: ORG
```

```
# Iterate through recognized entities and print their details
for ent in doc.ents:
    print(f"Entity: {ent.text}")
    print(f"Label: {ent.label_}")
    print(f"Start Position: {ent.start}")
    print(f"End Position: {ent.end}")
    print(f"Entity Type: {spacy.explain(ent.label_)}")
    print("")
```

```
Entity: Apple Inc.
Label: ORG
Start Position: 0
End Position: 2
Entity Type: Companies, agencies, institutions, etc.

Entity: Steve Jobs
Label: PERSON
Start Position: 5
End Position: 7
Entity Type: People, including fictional

Entity: Cupertino
Label: GPE
Start Position: 8
End Position: 9
Entity Type: Countries, cities, states

Entity: California
Label: GPE
Start Position: 10
End Position: 11
Entity Type: Countries, cities, states

Entity: Mac
Label: ORG
Start Position: 19
End Position: 20
Entity Type: Companies, agencies, institutions, etc.
```

Create a custom Named Entity Model

In spaCy, you can add custom entities that have not been identified in the pre-trained models by training your own Named Entity Recognition (NER) model. This involves providing labeled examples of your custom entities and then fine-tuning the model.

Here is an example of how to train a custom NER model in spaCy: https://spacy.io/usage/training#ner

```
from spacy.training.iob_utils import offsets_to_biluo_tags

# The text to annotate
text = "Apple Inc. was founded by Steve Jobs in Cupertino, California. It is known for its iPhone and Mac products."

# --- Add custom entities annotations manually ---
annotations = [
    (text.find("Apple Inc."), text.find("Apple Inc.") + len("Apple Inc."), "ORG"),
    (text.find("Steve Jobs"), text.find("Steve Jobs") + len("Steve Jobs"), "PERSON"),
    (text.find("Cupertino, California"), text.find("Cupertino, California") + len("Cupertino, California"), "GPE")]

# --- Step 2: Check entity alignment ---
doc = nlp.make_doc(text) #splitted to tokens
tags = offsets_to_biluo_tags(doc, annotations) #converts annotations into BILUO tags (Begin, Inside, Last, Unit, Outside).
print("Alignment Tags:", tags[:10])
```

```
Alignment Tags: ['B-ORG', 'L-ORG', 'O', 'O', 'O', 'B-PERSON', 'L-PERSON', 'O', 'B-GPE', 'I-GPE']
```

Visualization

```
# Preview NER visualization with the pre-trained model.
# The actual custom model will be trained and loaded in Task 3 below.
from spacy import displacy

doc2 = nlp(text)
displacy.render(doc2, style='ent', jupyter=True)
```

Apple Inc. **ORG** was founded by Steve Jobs **PERSON** in Cupertino **GPE** , California **GPE** . It is known for its iPhone and Mac **ORG** products.

Task 1

1. Ask a user to enter a sentence, and identify the named entities in the sentence.
2. Print the details of each entity in the sentence provided by the user.

```
# Task 1: Ask for a sentence and print the detected entity details.
# If the input is empty, use a sample sentence (helpful for testing).
try:
    user_sentence = input('Enter an English sentence for NER (Enter for example): ').strip()
except EOFError:
    user_sentence = ''

if not user_sentence:
```

```
    user_sentence = 'Bill Gates visited Microsoft in Seattle on June 10, 2024.'

user_doc = nlp(user_sentence)
print('\nSentence:', user_sentence)

if not user_doc.ents:
    print('No named entities detected.')
else:
    for ent in user_doc.ents:
        print(f'Entity: {ent.text}')
        print(f'Label: {ent.label_}')
        print(f'Type: {spacy.explain(ent.label_)}')
        print(f'Start position (character): {ent.start_char}')
        print(f'End position (character): {ent.end_char}')
        print('-' * 40)
```

```
Enter an English sentence for NER (Enter for example): Faisal visited Google in London on October 10, 2026.

Sentence: Faisal visited Google in London on October 10, 2026.
Entity: Faisal
Label: PERSON
Type: People, including fictional
Start position (character): 0
End position (character): 6
----------------------------------------
Entity: Google
Label: ORG
Type: Companies, agencies, institutions, etc.
Start position (character): 15
End position (character): 21
----------------------------------------
Entity: London
Label: GPE
Type: Countries, cities, states
Start position (character): 25
End position (character): 31
----------------------------------------
Entity: October 10, 2026
Label: DATE
Type: Absolute or relative dates or periods
Start position (character): 35
End position (character): 51
----------------------------------------
```

Task 2

Use the paragraph below to answer the following questions:

“This agreement is made and entered into by and between ‘Abc & Co.’ and ‘Bcd LLC’ for term 1 year starting from April 1, 2020, hereinafter collectively referred to as the Parties. Samuel P. Jackson in the place (New York) and on the date written below, with the

following terms and conditions."

1. Find any entity that has not been identified and add it.
2. Find the count of 'PERSON' and 'ORG' in the paragraph above.
3. Visualize the entities in the paragraph.

```
# Task 2: Detect missing entities, add them, count PERSON/ORG, and visualize.
from collections import Counter
from spacy.tokens import Span
from spacy import displacy

paragraph = ("This agreement is made and entered into by and between 'Abc & Co.'  "
             "and 'Bcd LLC' for term 1 year starting from April 1, 2020,  "
             "hereinafter collectively referred to as the Parties. Samuel P. Jackson "
             "in the place (New York) and on the date written below, "
             "with the following terms and conditions.")

agreement_doc = nlp(paragraph)
print('Entities detected by spaCy before adding missing ones:')
for ent in agreement_doc.ents:
    print(f'  {ent.text:<25} {ent.label_}')

# Expected entities in this agreement. Add any entities the model missed
# (or identified only partially) while keeping unrelated model predictions.
expected_entities = [
    ('Abc & Co.', 'ORG'),
    ('Bcd LLC', 'ORG'),
    ('April 1, 2020', 'DATE'),
    ('Samuel P. Jackson', 'PERSON'),
    ('New York', 'GPE'),
]

manual_spans = []
for entity_text, label in expected_entities:
    start = paragraph.find(entity_text)
    if start == -1:
        raise ValueError(f'Expected text not found: {entity_text}')
    end = start + len(entity_text)
    span = agreement_doc.char_span(start, end, label=label, alignment_mode='strict')
    if span is None:
        raise ValueError(f'Entity is not aligned with tokens: {entity_text}')
    if not any(ent.start == span.start and ent.end == span.end
               and ent.label_ == label for ent in agreement_doc.ents):
        manual_spans.append(span)

# Remove existing partial/incorrect spans only where they overlap a correction.
kept_spans = [
    ent for ent in agreement_doc.ents
```

```
    for ent in agreement_doc.ents
    if not any(ent.start < span.end and span.start < ent.end
               for span in manual_spans)
]
agreement_doc.set_ents(sorted(kept_spans + manual_spans, key=lambda span: span.start))

print('\nMissing/corrected entities added:')
if manual_spans:
    for span in manual_spans:
        print(f'  {span.text:<25} {span.label_}')
else:
    print('  None (all expected entities were correctly recognized).')

print('\nFinal named entities:')
for ent in agreement_doc.ents:
    print(f'  {ent.text:<25} {ent.label_}')

entity_counts = Counter(ent.label_ for ent in agreement_doc.ents)
print('\nPERSON count:', entity_counts['PERSON'])
print('ORG count:', entity_counts['ORG'])

# Visualize the completed annotations.
displacy.render(agreement_doc, style='ent', jupyter=True)
```

```
Entities detected by spaCy before adding missing ones:
  between ‘Abc & Co.        ORG
  1 year                    DATE
  April 1, 2020             DATE
  Samuel P. Jackson         PERSON
  New York                  GPE

Missing/corrected entities added:
  Abc & Co.                 ORG
  Bcd LLC                   ORG

Final named entities:
  Abc & Co.                 ORG
  Bcd LLC                   ORG
  1 year                    DATE
  April 1, 2020             DATE
  Samuel P. Jackson         PERSON
  New York                  GPE

PERSON count: 1
ORG count: 2
```

This agreement is made and entered into by and between ‘ Abc & Co. **ORG** ’ and ‘ Bcd LLC **ORG** ’ for term 1 year **DATE** starting from April 1, 2020 **DATE** , hereinafter collectively referred to as the Parties. Samuel P. Jackson **PERSON** in the place (New York **GPE**) and on the date written below, with the following terms and conditions.

Task 3: Training a Custom NER Model

text = "Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. It is known for its Windows operating system and Office software products."

1. Create a training example using `Example.from_dict()` with the provided text and annotations.
2. Train the NER model for **20 iterations**
3. Save the trained model in a folder named `custom_ner_model`.
4. Load the saved model and use it to identify named entities in a sentence.

```
# Task 3 — Improved Custom NER Model (exactly 20 iterations)
# Student Name: Faisal AL Zahrani
# Student ID: 2230000363
import random
from pathlib import Path

import spacy
from spacy.training import Example
from spacy.util import minibatch, fix_random_seed
from spacy import displacy

# Use a fixed seed for more reproducible training.
fix_random_seed(42)

# The exact required sentence from Lab 9.
training_text = (
    'Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. '
    'It is known for its Windows operating system and Office software products.'
)

# Training examples: the assigned sentence + short, varied examples.
# Additional examples help the model learn PRODUCT entities (Windows, Office).
examples_specs = [
    (training_text, [
        ('Microsoft', 'ORG'), ('Bill Gates', 'PERSON'),
        ('Paul Allen', 'PERSON'), ('Albuquerque', 'GPE'),
        ('New Mexico', 'GPE'), ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')
    ]),
    ('Microsoft sells Windows and Office worldwide.', [
        ('Microsoft', 'ORG'), ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')
    ]),
    ('Bill Gates and Paul Allen founded Microsoft.', [
        ('Bill Gates', 'PERSON'), ('Paul Allen', 'PERSON'), ('Microsoft', 'ORG')
    ]),
    ('Windows is an operating system developed by Microsoft.', [
        ('Windows', 'PRODUCT'), ('Microsoft', 'ORG')
```

```
        ]),
        ('Office is a productivity suite created by Microsoft.', [
            ('Office', 'PRODUCT'), ('Microsoft', 'ORG')
        ]),
        ('Albuquerque is a city in New Mexico.', [
            ('Albuquerque', 'GPE'), ('New Mexico', 'GPE')
        ]),
        ('Paul Allen met Bill Gates in Albuquerque.', [
            ('Paul Allen', 'PERSON'), ('Bill Gates', 'PERSON'), ('Albuquerque', 'GPE')
        ]),
        ('I installed Windows yesterday and opened Office today.', [
            ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')
        ]),
        ('Microsoft introduced Windows and Office products.', [
            ('Microsoft', 'ORG'), ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')
        ]),
        ('Bill Gates helped grow Microsoft with Paul Allen.', [
            ('Bill Gates', 'PERSON'), ('Microsoft', 'ORG'), ('Paul Allen', 'PERSON')
        ]),
    ]

    TRAIN_DATA = []
    for sentence, labeled_words in examples_specs:
        entity_offsets = []
        for entity_text, label in labeled_words:
            start = sentence.find(entity_text)
            if start < 0:
                raise ValueError(f'Entity not found in training sentence: {entity_text}')
            entity_offsets.append((start, start + len(entity_text), label))
        TRAIN_DATA.append((sentence, {'entities': entity_offsets}))

    # 1. Create the custom model and Example.from_dict() annotations.
    custom_nlp = spacy.blank('en')
    ner = custom_nlp.add_pipe('ner')
    for _, annotation in TRAIN_DATA:
        for _, _, label in annotation['entities']:
            ner.add_label(label)

    def create_example(sentence, annotation):
        return Example.from_dict(custom_nlp.make_doc(sentence), annotation)

    training_example = create_example(*TRAIN_DATA[0])
    print('Entities in the required annotated example:')
    for ent in training_example.reference.ents:
        print(f'  {ent.text:<22} {ent.label_}')

    optimizer = custom_nlp.initialize(
        get_examples=lambda: [create_example(s, a) for s, a in TRAIN_DATA]
    )
```

```
# 2. Train for EXACTLY 20 iterations (epochs), using small batches.
for epoch in range(20):
    losses = {}
    shuffled_data = TRAIN_DATA.copy()
    random.Random(42 + epoch).shuffle(shuffled_data)
    for batch in minibatch(shuffled_data, size=2):
        examples = [create_example(sentence, annotation)
                    for sentence, annotation in batch]
        custom_nlp.update(examples, sgd=optimizer, drop=0.05, losses=losses)
    print(f'Iteration {epoch + 1:02d}/20 | NER loss: {losses.get("ner", 0):.4f}')

# 3. Save the trained model using the lab's required folder name.
model_dir = Path('custom_ner_model')
custom_nlp.to_disk(model_dir)
print('\nModel saved to:', model_dir.resolve())

# 4. Reload the model and test entity recognition.
loaded_nlp = spacy.load(str(model_dir))
test_sentence = (
    'Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. '
    'It is known for Windows and Office.'
)
result_doc = loaded_nlp(test_sentence)
print('\nEntities detected by the saved model:')
for ent in result_doc.ents:
    print(f'  {ent.text:<22} {ent.label_}')

expected = {
    'Microsoft': 'ORG', 'Bill Gates': 'PERSON', 'Paul Allen': 'PERSON',
    'Albuquerque': 'GPE', 'New Mexico': 'GPE',
    'Windows': 'PRODUCT', 'Office': 'PRODUCT'
}
found = {ent.text: ent.label_ for ent in result_doc.ents}
print(f'\nCorrectly identified target entities: '
      f'{sum(found.get(name) == label for name, label in expected.items())}'
      f'/{len(expected)}')

# A second sentence for a quick sanity check (not a formal evaluation).
extra_doc = loaded_nlp('Microsoft launched Windows and Office.')
print('\nExtra test:', [(ent.text, ent.label_) for ent in extra_doc.ents])

displacy.render(result_doc, style='ent', jupyter=True)
```

```
Entities in the required annotated example:
  Microsoft             ORG
  Bill Gates            PERSON
  Paul Allen            PERSON
  Albuquerque           GPE
  New Mexico            GPE
  Windows               PRODUCT
  Office                PRODUCT
Iteration 01/20 | NER loss: 88.4760
Iteration 02/20 | NER loss: 75.4288
Iteration 03/20 | NER loss: 47.4837
Iteration 04/20 | NER loss: 34.4309
Iteration 05/20 | NER loss: 49.0310
Iteration 06/20 | NER loss: 42.2465
Iteration 07/20 | NER loss: 14.2484
Iteration 08/20 | NER loss: 4.6594
Iteration 09/20 | NER loss: 5.1902
Iteration 10/20 | NER loss: 5.0188
Iteration 11/20 | NER loss: 1.4310
Iteration 12/20 | NER loss: 0.2397
Iteration 13/20 | NER loss: 0.0023
Iteration 14/20 | NER loss: 0.0000
Iteration 15/20 | NER loss: 0.0000
Iteration 16/20 | NER loss: 0.0000
Iteration 17/20 | NER loss: 0.0000
Iteration 18/20 | NER loss: 0.0000
Iteration 19/20 | NER loss: 0.0000
Iteration 20/20 | NER loss: 0.0000

Model saved to: /content/custom_ner_model

Entities detected by the saved model:
  Microsoft             ORG
  Bill Gates            PERSON
  Paul Allen            PERSON
  Albuquerque           GPE
  New Mexico            GPE
  Windows               PRODUCT
  Office                PRODUCT

Correctly identified target entities: 7/7

Extra test: [('Microsoft', 'ORG'), ('Windows', 'PRODUCT'), ('Office', 'PRODUCT')]
```

Microsoft **ORG** was founded by Bill Gates **PERSON** and Paul Allen **PERSON** in Albuquerque **GPE** , New Mexico **GPE** . It is known for Windows **PRODUCT** and Office **PRODUCT**

.